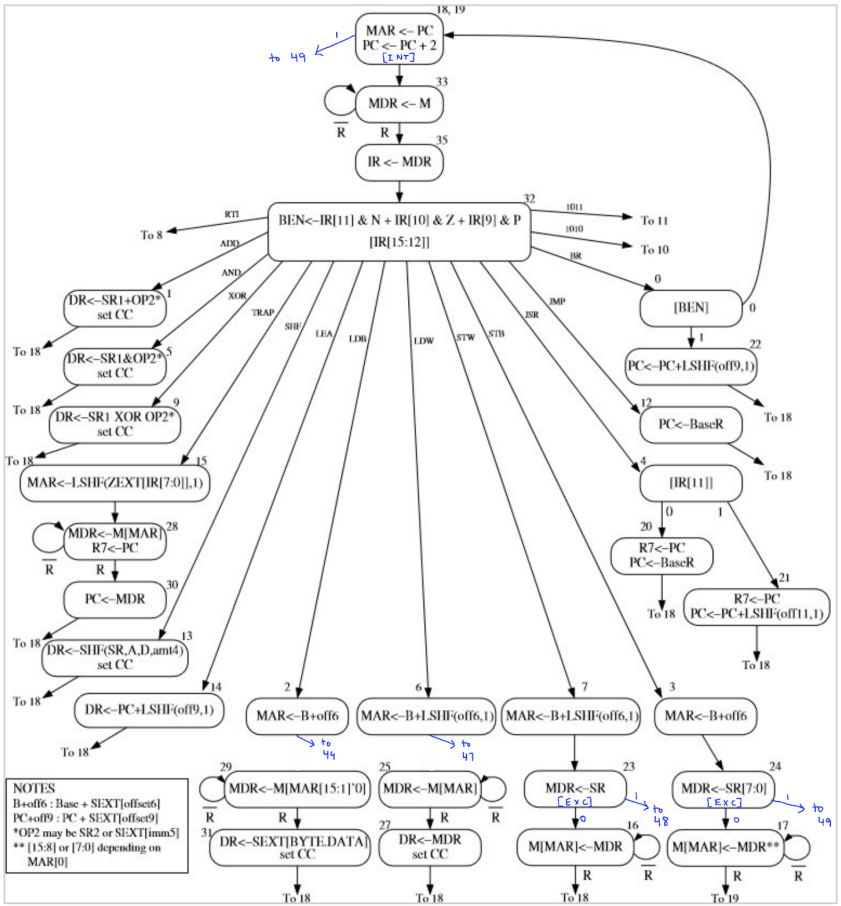
EE460N Lab 4: Interrupts and Exceptions

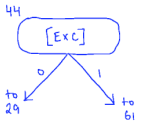
This document covers the changes made to the LC-3b microarchitecture to support the timer interrupt, the protection, unaligned access, and unknown opcode exceptions, and the RTI instruction. Interrupt and exception initiation share one microcode sequence: every entry point backs up the PC, saves the PSR, switches to supervisor mode and the supervisor stack, pushes PSR and PC, then loads the PC from the vector table at x0200 + (EXCV << 1). The control store stays at 64 rows.

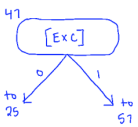
1. State Diagram Changes

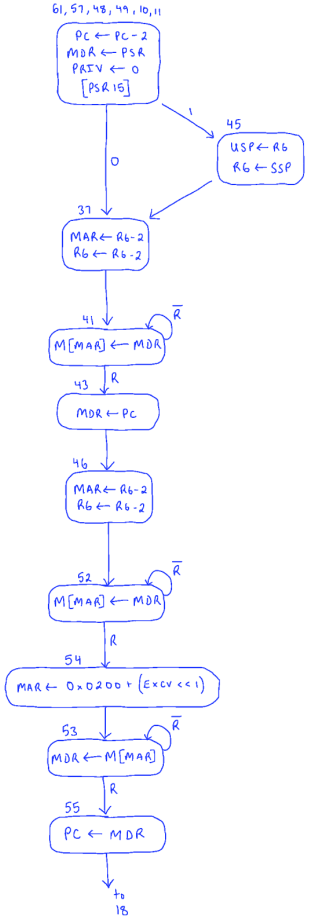
New states and arcs are drawn in blue on the diagram below. New state numbers are shown above each box; the six-bit encoding is the binary form of that number (listed in the table).

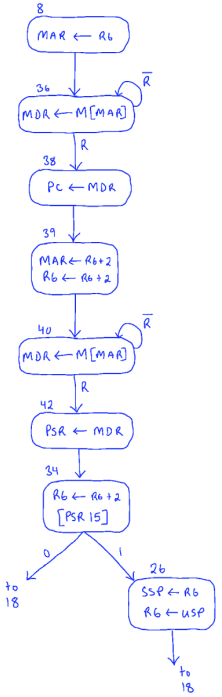
Figure 1 (next page, rotated for size). Modified state diagram, changes in blue.


18, 19
MAR <- PC
PC <- PC + 2
[INT]
to 49
1
33
MDR <- M
R
R
35
IR <- MDR
32
BEN<-IR[11] & N + IR[10] & Z + IR[9] & P
[IR[15:12]]
RTI
To 8
ADD
AND
XOR
TRAP
SHF
LEA
LDB
LDW
STW
STB
JSR
JMP
1011
To 11
1010
To 10
BR
0
[BEN]
0
1
22
PC<-PC+LSHF(off9,1)
To 18
12
PC<-BaseR
To 18
4
[IR[11]]
0
1
20
R7<-PC
PC<-BaseR
To 18
21
R7<-PC
PC<-PC+LSHF(off11,1)
To 18
1
DR<-SR1+OP2*
set CC
To 18
5
DR<-SR1&OP2*
set CC
To 18
9
DR<-SR1 XOR OP2*
set CC
To 18
15
MAR<-LSHF(ZEXT[IR[7:0]],1)
28
MDR<-M[MAR]
R7<-PC
R
R
30
PC<-MDR
To 18
13
DR<-SHF(SR,A,D,amt4)
set CC
To 18
14
DR<-PC+LSHF(off9,1)
To 18
2
MAR<-B+off6
to 44
6
MAR<-B+LSHF(off6,1)
to 47
7
MAR<-B+LSHF(off6,1)
3
MAR<-B+off6
29
MDR<-M[MAR[15:1]'0]
R
R
31
DR<-SEXT[BYTE.DATA]
set CC
To 18
25
MDR<-M[MAR]
R
R
27
DR<-MDR
set CC
To 18
23
MDR<-SR
[EXC]
1
to 48
0
16
M[MAR]<-MDR
R
R
To 18
24
MDR<-SR[7:0]
[EXC]
1
to 49
0
17
M[MAR]<-MDR**
R
R
To 19
NOTES
B+off6 : Base + SEXT[offset6]
PC+off9 : PC + SEXT[offset9]
*OP2 may be SR2 or SEXT[imm5]
** [15:8] or [7:0] depending on MAR[0]


44
[EXC]
0
1
to 29
to 61


47
[EXC]
0
1
to 25
to 57


61, 57, 48, 49, 10, 11
PC ← PC - 2
MDR ← PSR
PRIV ← 0
[PSR 15]
0
1
45
USP ← R6
R6 ← SSP
37
MAR ← R6 - 2
R6 ← R6 - 2
41
M[MAR] ← MDR
R
43
MDR ← PC
46
MAR ← R6 - 2
R6 ← R6 - 2
52
M[MAR] ← MDR
R
54
MAR ← 0x0200 + (EXCV << 1)
53
MDR ← M[MAR]
R
55
PC ← MDR
to 18


8
MAR ← R6
36
MDR ← M[MAR]
R
38
PC ← MDR
39
MAR ← R6 + 2
R6 ← R6 + 2
40
MDR ← M[MAR]
R
42
PSR ← MDR
34
R6 ← R6 + 2
[PSR 15]
0
1
26
SSP ← R6
R6 ← USP
to 18
to 18

1.1 Modified existing states

State	Change	Next state
18, 19 (010010, 010011)	Now check INT (COND = 101) while doing MAR<-PC, PC<-PC+2	INT=0: 33. INT=1: 49
2 (LDB)	Next state changed from 29 to new check state 44	44
6 (LDW)	Next state changed from 25 to new check state 47	47
23 (STW, MDR<-SR)	Now checks EXC (COND = 110), DATA.SIZE = WORD	EXC=0: 16. EXC=1: 48
24 (STB, MDR<-SR[7:0])	Now checks EXC (COND = 110), DATA.SIZE = BYTE	EXC=0: 17. EXC=1: 49
10, 11 (unknown opcodes)	Previously unused. Now the start of exception initiation (vector x04)	PSR15=0: 37. PSR15=1: 45
8 (RTI)	Now the first state of RTI	36

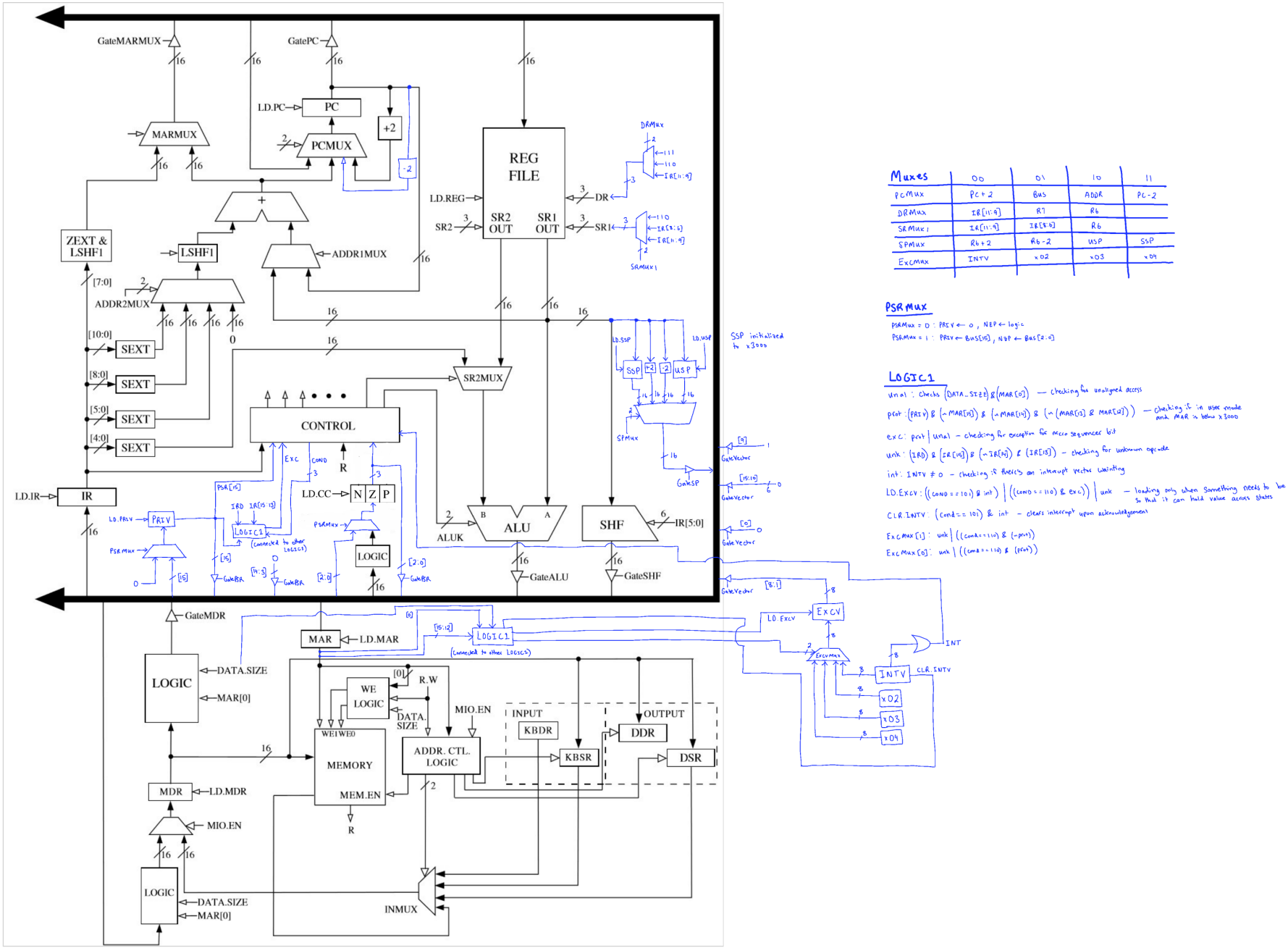
1.2 New states

State	Encoding	Operation	Next state
44	101100	Exception check for LDB (DATA.SIZE = BYTE, so only protection can fire)	EXC=0: 29. EXC=1: 61
47	101111	Exception check for LDW (DATA.SIZE = WORD)	EXC=0: 25. EXC=1: 57
48, 49, 57, 61 (and 10, 11)	110000, 110001, 111001, 111101 (001010, 001011)	PC<-PC-2; MDR<-PSR; PRIV<-0. Identical entry states for each path into initiation. PC-2 undoes the fetch increment so the faulting instruction (or the interrupted fetch) re-executes after RTI.	PSR15=0: 37. PSR15=1: 45
45	101101	USP<-R6; R6<-SSP (stack switch, only taken if we came from user mode)	37
37	100101	MAR<-R6-2; R6<-R6-2	41
41	101001	M[MAR]<-MDR (push old PSR), wait for R	R=0: 41. R=1: 43
43	101011	MDR<-PC	46
46	101110	MAR<-R6-2; R6<-R6-2	52
52	110100	M[MAR]<-MDR (push PC), wait for R	R=0: 52. R=1: 54
54	110110	MAR<-x0200 + (EXCV << 1)	53
53	110101	MDR<-M[MAR] (read handler address), wait for R	R=0: 53. R=1: 55
55	110111	PC<-MDR	18
36	100100	RTI: MDR<-M[MAR] (MAR = R6 from state 8), wait for R	R=0: 36. R=1: 38
38	100110	PC<-MDR (pop PC)	39
39	100111	MAR<-R6+2; R6<-R6+2	40
40	101000	MDR<-M[MAR], wait for R	R=0: 40. R=1: 42
42	101010	PSR<-MDR: PRIV<-MDR[15], N/Z/P<-MDR[2:0] (PSRMUX = 1)	34
34	100010	R6<-R6+2; check restored PSR[15]	PSR15=0: 18. PSR15=1: 26
26	011010	SSP<-R6; R6<-USP (switch back to user stack)	18

Design notes. Exceptions are checked in the cycle after MAR is loaded and before any memory access starts (states 44, 47, 23, 24), so a faulting access never touches memory (clarification 11). The unknown opcode case needs no extra check state: state 32 already dispatches to 10 or 11 on opcode 1010/1011. TRAP (states 15, 28) does not check EXC, so user programs can read the trap vector table (clarification 13). Nested interrupts and exceptions work because state 45 is skipped when PSR[15] is already 0, so a nested entry keeps pushing onto the supervisor stack.

2. Datapath Changes

Additions are drawn in blue. Figure 2 is on the next page, rotated for size.



Muxes	00	01	10	11
PCMux	PC+2	Bus	ADDR	PC-2
DRMux	IR[11:9]	R7	R6	
SR1Mux	IR[11:9]	IR[8:6]	R6	
SPMux	R6+2	R6-2	USP	SSP
ExcMux	INTV	x02	x03	x04

PSRMux

PSRMux = 0 : PRIV ← 0 , N,Z,P ← logic
PSRMux = 1 : PRIV ← Bus[15] , N,Z,P ← Bus[2:0]

LOGIC1

unal : Checks (DATA_SIZE) & (MAR[0]) — Checking for unaligned access
priv : (PRIV) & (~MAR[15]) & (~MAR[14]) & (~(MAR[13] & MAR[12])) — checking if in user mode and MAR is below x3000
exc : priv | unal — checking for exception for micro sequencer bit
unk : (IRD) & (IR[15]) & (~IR[14]) & (IR[13]) — checking for unknown opcode
int : INTV ≠ 0 — checking if there's an interrupt vector waiting
LD.EXCV : ((cond == 101) & int) | ((cond == 110) & exc) | unk — loading only when something needs to be so that it can hold value across states
CLR.INTV : (cond == 101) & int — clears interrupt upon acknowledgement
ExcMux[1] : unk | ((cond == 110) & (~priv))
ExcMux[0] : unk | ((cond == 110) & (priv))

Figure 2: Modified datapath (additions in blue)

2.1 New structures

Structure	Control	Purpose
PRIV register (PSR[15])	LD.PRIV, PSRMUX	Privilege bit. PSRMUX=0 loads 0 (enter supervisor). PSRMUX=1 loads BUS[15] (RTI restore). Initialized to 1 (user).
PSRMUX on N/Z/P input	PSRMUX, LD.CC	PSRMUX=0: normal setcc logic from the bus. PSRMUX=1: load N/Z/P directly from BUS[2:0] when RTI restores the PSR.
GatePSR tristate	GatePSR	Drives {PRIV, 12'b0, N, Z, P} onto the bus so the PSR can be latched into MDR.
SSP and USP registers	LD.SSP, LD.USP	Hold the inactive stack pointer. Loaded from SR1 OUT (R6). SSP initialized to x3000.
SPMUX and GateSP	SPMUX[1:0], GateSP	Selects R6+2, R6-2, USP, or SSP (R6 comes from SR1 OUT) and drives it on the bus, used for push/pop and stack switching.
DRMUX widened to 2 bits	DRMUX[1:0]	Adds R6 as a destination (00 IR[11:9], 01 R7, 10 R6).
SR1MUX widened to 2 bits	SR1MUX[1:0]	Adds R6 as a source (00 IR[11:9], 01 IR[8:6], 10 R6).
PCMUX input 11: PC-2	PCMUX[1:0]	Backs the PC up by 2 so the interrupted or faulting instruction re-executes.
INTV register (8 bits)	set by timer, CLR.INTV	Holds a pending interrupt vector. Nonzero means INT. Cleared when the interrupt is taken.
EXCV register (8 bits) and ExcMux	LD.EXCV, ExcMux[1:0]	Latches the vector for the event being taken: INTV, x02, x03, or x04.
Vector address logic and GateVector	GateVector	Drives x0200 + (EXCV << 1), i.e. {7'b0000001, EXCV, 1'b0}, onto the bus to address the vector table.
LOGIC1	inputs: MAR, DATA.SIZE, PRIV, IRD, IR[15:13], INTV, COND	Generates EXC, INT, unk, LD.EXCV, ExcMux, and CLR.INTV (equations below).

2.2 LOGIC1 equations

Signal	Equation	Meaning
unal	DATA.SIZE & MAR[0]	Word access to an odd address
prot	PRIV & (MAR < x3000), i.e. PRIV & ~MAR[15] & ~MAR[14] & ~(MAR[13] & MAR[12])	User mode touching system space
EXC	prot | unal	Exception bit sent to the microsequencer
unk	IRD & IR[15] & ~IR[14] & IR[13]	Opcode 1010 or 1011 being decoded
INT	INTV != 0	Interrupt pending
LD.EXCV	(COND==101 & INT) | (COND==110 & EXC) | unk	Latch EXCV only when an event is actually taken
ExcMux[1]	unk | (COND==110 & ~prot)	00 INTV, 01 x02, 10 x03, 11 x04
ExcMux[0]	unk | (COND==110 & prot)	Protection beats unaligned when both are true
CLR.INTV	COND==101 & INT	Acknowledge the interrupt when it is taken

3. New Control Signals

The microinstruction grew from 35 to 47 bits. Bit order matches enum CS_BITS in lc3bsim4.c: IRD, COND[2:0], J[5:0], the original datapath signals, then GatePSR, GateSP, GateVector, SPMUX[1:0], LD.USP, LD.SSP, LD.PRIV, PSRMUX, DRMUX[1:0], SR1MUX[1:0].

Signal	Values	Use
COND[2] (COND now 3 bits)	000 uncond, 001 ready, 010 branch, 011 addr mode, 100 PSR[15], 101 INT, 110 EXC	Three new branch conditions. 100 in states 10/11/48/49/57/61 and 34; 101 in 18/19; 110 in 44/47/23/24.
GatePSR	NO, YES	PSR onto bus (states 10, 11, 48, 49, 57, 61)
GateSP	NO, YES	SPMUX output onto bus (states 26, 34, 37, 39, 45, 46)
GateVector	NO, YES	x0200 + (EXCV << 1) onto bus (state 54)

Signal	Values	Use
SPMUX[1:0]	00 R6+2, 01 R6-2, 10 USP, 11 SSP	Stack pointer arithmetic and swap
LD.USP	NO, LOAD	Save R6 into USP on entry from user mode (state 45)
LD.SSP	NO, LOAD	Save R6 into SSP on return to user mode (state 26)
LD.PRIV	NO, LOAD	Load PSR[15] (entry states, and state 42)
PSRMUX	0 entry/setcc, 1 restore from bus	Chooses PRIV<-0 vs PRIV<-BUS[15], and normal setcc vs N/Z/P<-BUS[2:0]
DRMUX[1:0]	00 IR[11:9], 01 R7, 10 R6	Widened from 1 bit to allow writing R6
SR1MUX[1:0]	00 IR[11:9], 01 IR[8:6], 10 R6	Widened from 1 bit to allow reading R6
PCMUX[1:0]	new value 11 = PC-2	Existing field, new input

4. Microsequencer Changes

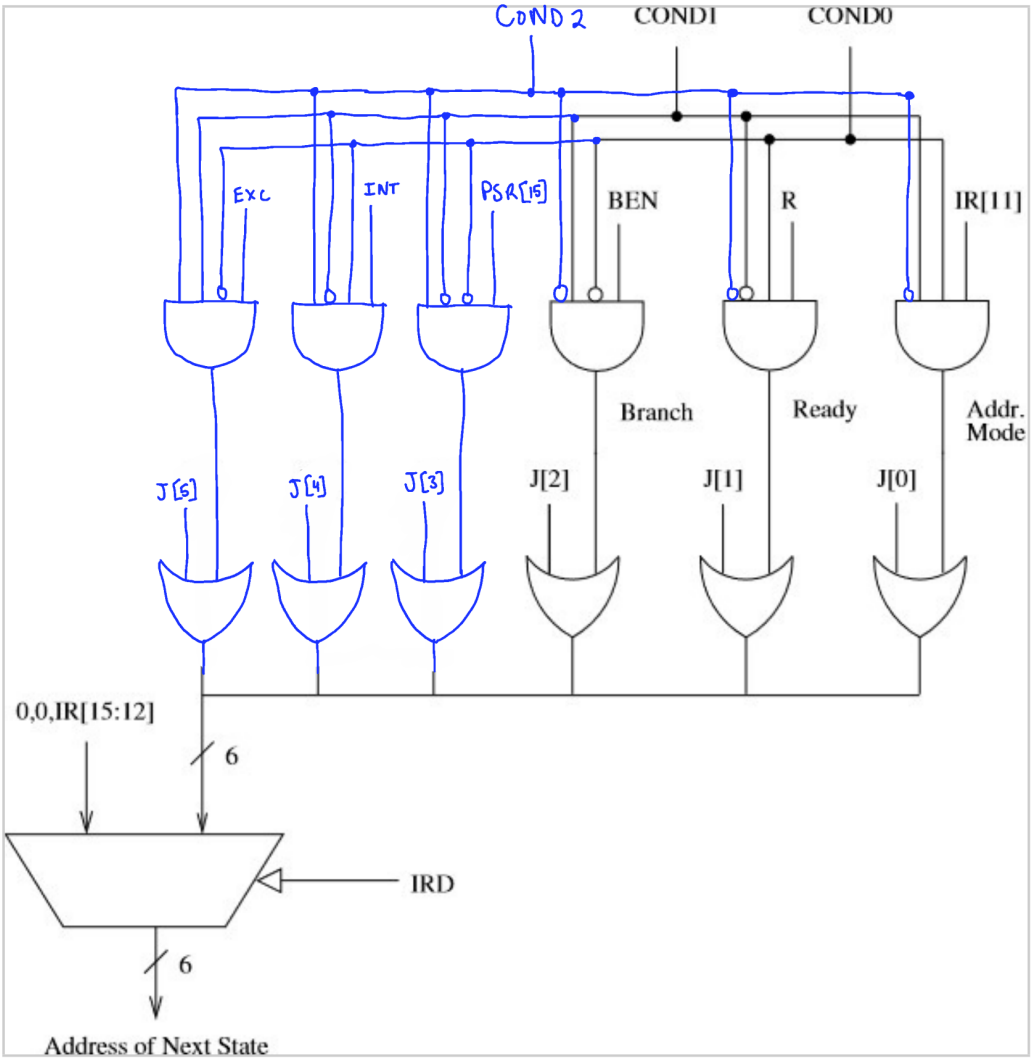


Figure 3. Modified microsequencer (additions in blue)

COND was widened from 2 to 3 bits so three new conditions could be added without touching the original four. The original Branch, Ready, and Addr. Mode AND gates now also require COND2 = 0. Three new AND gates each decode one COND value and OR their result into a J bit that is zero in the base address:

Condition	COND	Gate output	OR'd into	Why that bit
PSR[15]	100	COND2 & ~COND1 & ~COND0 & PSR[15]	J[3]	37 to 45 on entry from user mode; 18 to 26 on RTI back to user mode
INT	101	COND2 & ~COND1 & COND0 & INT	J[4]	33 to 49 (18/19 jump to interrupt initiation instead of fetch)
EXC	110	COND2 & COND1 & ~COND0 & EXC	J[5]	29/25/16/17 to 61/57/48/49 (exception initiation)

The IRD mux is unchanged, so opcodes 1010 and 1011 still go from state 32 to states 10 and 11, which now begin exception initiation. The EXC, INT, and PSR[15] inputs come from LOGIC1 and the PRIV register in the datapath. The interrupt is only sampled in states 18/19, at an instruction boundary. Exceptions are sampled in the state right after MAR is loaded for a load or store, before the memory access begins.

5. Verification

Test	Result
Protection (STW to x0000)	Cycle 299: x4000 = 1. Cycle 513: ISR returned to x301A, x4000 = 2, R6 restored. Halt at cycle 1459 via x1600, xC014 = x0052.
Unaligned (STW to xC017)	Halt at cycle 1459 via x1A00, R1 = xC017, x4000 = 2.
Unknown opcode (.FILL xA000)	Halt via x1C00 (R7 = x1C02).
Handler that returns with RTI	Control returns to user mode with R6 = user stack value and PRIV = 1.
grader2 test run	100